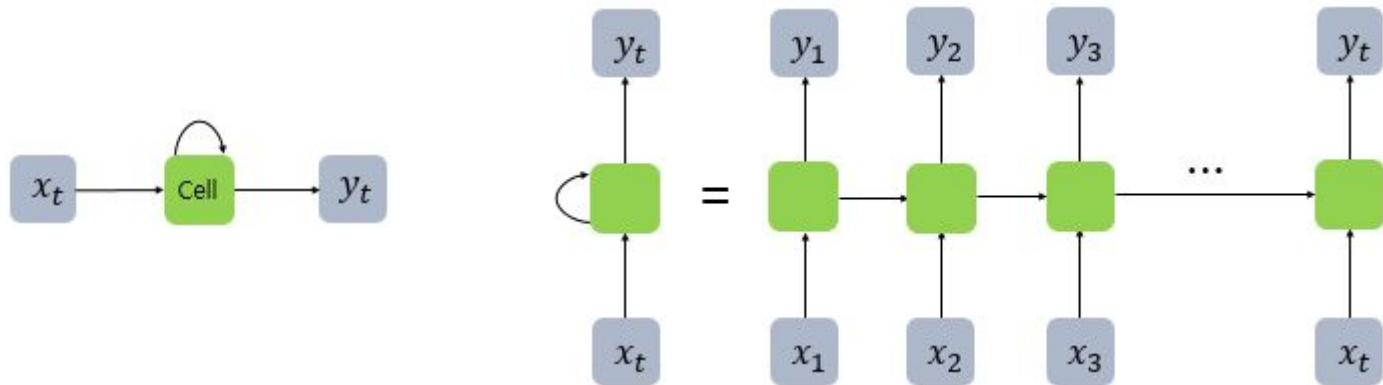


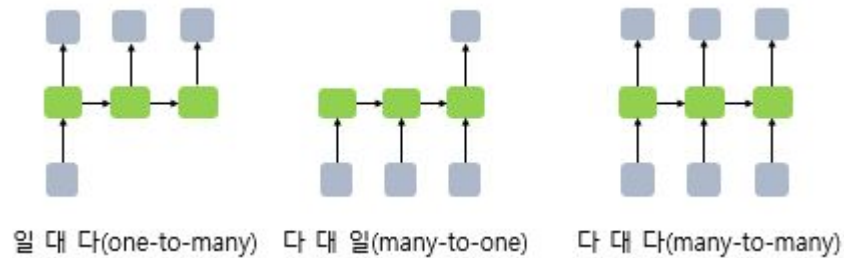
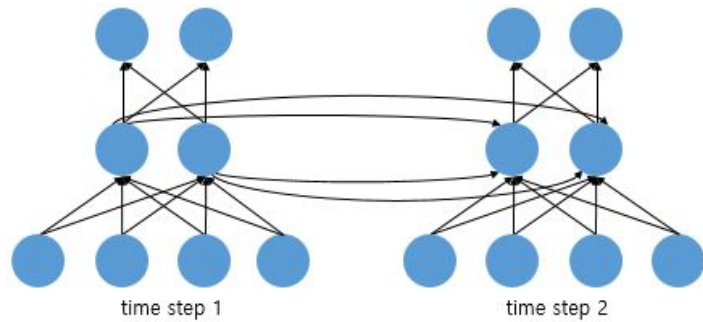
Vision Transformers

JD-edu

순환 신경망



순환 신경망

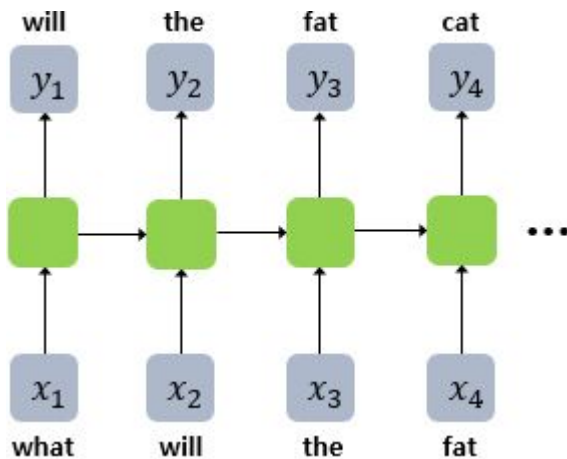


순환 신경망의 활용 - 스팸메일 분류



순환 신경망의 활용 - 언어모델

- 입력: what will the fat cat sit on
- 출력: will the fact cat
- 교사 강요



워드 임베딩

- 단어를 벡터로 표현 하는 것
- 원-핫-임베딩
 - 단어가 늘어나면 벡터가 커짐

$V = ["I", "love", "robot", "AI"]$

단어	원-핫 벡터
I	[1, 0, 0, 0]
love	[0, 1, 0, 0]
robot	[0, 0, 1, 0]
AI	[0, 0, 0, 1]

워드 임베딩 - 밀집 표현(Dense Representation)

- 희소표현

강아지 = [0 0 0 0 1 0 0 0 0 0 0 0 ... 중략 ... 0]

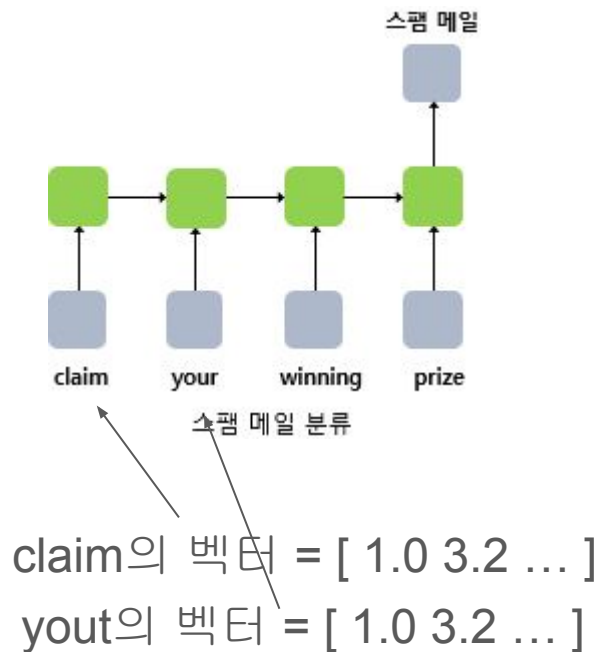
이때 1 뒤의 0의 수는 9995개. 차원은 10,000

- 밀집표현

강아지 = [0.2 1.8 1.1 -2.1 1.1 2.8 ... 중략 ...]

이 벡터의 차원은 128

- 밀집표현은 트레이닝 단계에서 학습이 되어야 함



토큰

✓ 2. 자연어(NLP)에서 토큰의 의미

자연어에서 토큰은 다음 중 하나일 수 있습니다:

✓ (1) 단어(word) 토큰

text

📄 코드 복사

```
"I love robots"  
→ ["I", "love", "robots"]
```

✓ (2) 서브워드(subword) 토큰 (BPE, WordPiece)

예를 들어 BERT 계열:

text

📄 코드 복사

```
"robotics" → ["robot", "##ics"]
```

✓ (3) 문자(character) 토큰

text

📄 코드 복사

```
"AI" → ["A", "I"]
```


단어 -> 토큰 -> 정수 ID -> 임베딩...

처리 흐름 요약

text

 코드 복사

토큰 → 정수 ID → 임베딩 벡터 → (여기서부터 의미가 생성됨)

예:

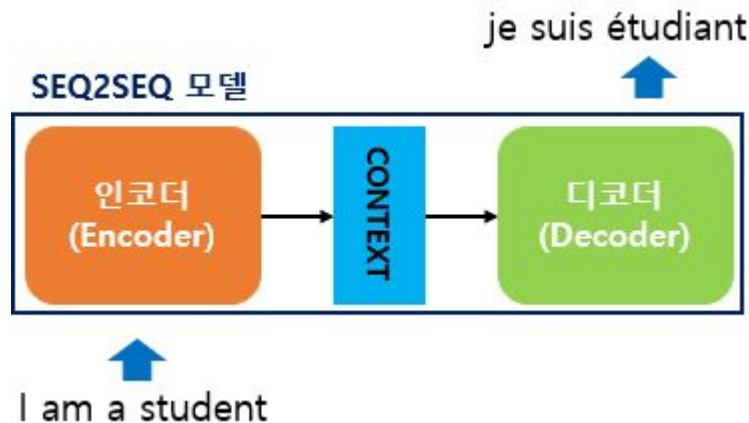
text

 코드 복사

"robot" → token_id = 523

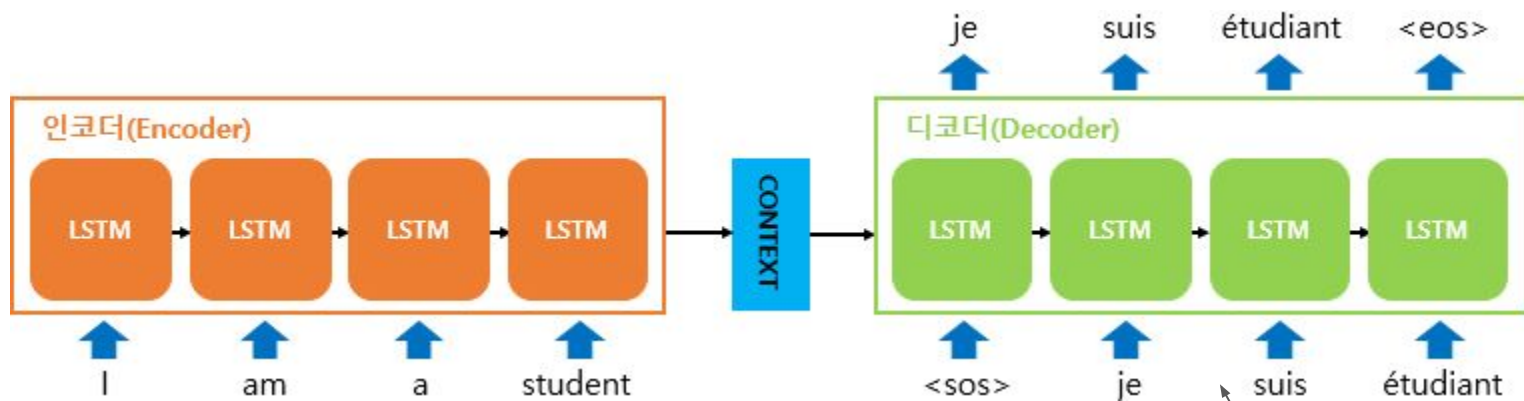
→ embedding[523] = [0.12, -0.4, 1.02, ...]

RNN에서 인코더 디코더(seq2seq)



seq2seq는 크게 인코더와 디코더라는 두 개의 모듈로 구성됩니다. 인코더는 입력 문장의 모든 단어들을 순차적으로 입력받은 뒤에 마지막에 이 모든 단어 정보들을 압축해서 하나의 벡터로 만드는데, 이를 컨텍스트 벡터(context vector)라고 합니다. 입력 문장의 정보가 하나의 컨텍스트 벡터로 모두 압축되면 인코더는 컨텍스트 벡터를 디코더로 전송합니다. 디코더는 컨텍스트 벡터를 받아서 번역된 단어를 한 개씩 순차적으로 출력합니다.

좀 더 구체적인 seq2seq 엔코더 디코더



교사강요

RNN seq2seq 인코더 디코더 해설

✓ 전체 데이터 흐름 한 줄 요약

text

📄 코드 복사

문장 → 토큰화 → 정수 ID → 임베딩 벡터 → 인코더 LSTM → Context Vector
→ 디코더 LSTM → 출력 벡터 → Linear → Softmax → 다음 토큰 ID → 단어

이제 그림의 왼쪽(Encoder)부터 오른쪽(Decoder)까지 단계별로 정확하게 대응시켜 설명하겠습니다.

✓ 1 입력 문장 → 토큰(Token)

예시 (그림의 입력)

text

📄 코드 복사

"I am a student"

토큰화 결과

text

📄 코드 복사

["I", "am", "a", "student"]

- ✓ 여기서 각 단어 = 토큰 1개
- ✓ 아직 숫자도 아니고, 의미 벡터도 아님 → 단순 기호(symbol)

RNN seq2seq 인코더 디코더 해설

✓ 2 토큰 → 정수 ID (Vocabulary Lookup)

어휘 사전(vocab)이 다음과 같다고 가정하면:

text

📄 코드 복사

"I" → 7
"am" → 32
"a" → 5
"student" → 104

그러면 입력 시퀀스는:

text

📄 코드 복사

["I", "am", "a", "student"]
→ [7, 32, 5, 104]

- ✓ 이 정수 ID가 딥러닝 모델에 들어가는 실제 입력 인덱스
- ✓ 이 단계까지는 아직 의미 없음

RNN seq2seq 인코더 디코더 해설

✅ 3 정수 ID → 임베딩 벡터 (Embedding Layer)

임베딩 테이블:

$$E \in \mathbb{R}^{V \times d}$$

각 ID는 임베딩 벡터로 변환됩니다:

text

📄 코드 복사

```
7   → [0.2, -0.1, 1.3, ...]
32  → [-0.8, 0.4, 0.9, ...]
5   → [1.1, -0.2, 0.03, ...]
104 → [0.5, 0.7, -0.6, ...]
```

즉:

text

📄 코드 복사

정수 ID → ✅ 밀집(Dense) 임베딩 벡터

RNN seq2seq 인코더 디코더 해설



임베딩 벡터 → 인코더 LSTM (시간 순서 처리)

이제 그림의 왼쪽 주황색 LSTM 블록(Encoder) 로 들어갑니다.

입력 흐름:

text

코드 복사

$x_1 \rightarrow \text{LSTM} \rightarrow h_1$

$x_2 \rightarrow \text{LSTM} \rightarrow h_2$

$x_3 \rightarrow \text{LSTM} \rightarrow h_3$

$x_4 \rightarrow \text{LSTM} \rightarrow h_4$

수식:

$$h_t = \text{LSTM}(x_t, h_{t-1})$$

의미 누적 관점:

text

코드 복사

"I" $\rightarrow h_1 = \text{"나"}$

"am" $\rightarrow h_2 = \text{"나는 ~이다"}$

"a" $\rightarrow h_3 = \text{"나는 하나의 ~"}$

"student" $\rightarrow h_4 = \text{"나는 학생이다"}$

RNN seq2seq 인코더 디코더 해설

✓ 5 마지막 은닉 상태 → Context Vector (문장의 의미 압축)

그림의 CONTEXT 화살표에 해당합니다.

text

📄 코드 복사

Context Vector = h_4

✓ 전체 문장

"I am a student"

의 의미가 고정 길이 벡터 1개로 압축됨

이 벡터는:

- 디코더의 초기 **hidden state**
- 번역 전체의 의미 요약본

RNN seq2seq 인코더 디코더 해설(이후 생략)



디코더 입력 시작: <SOS> → 정수 ID → 임베딩

디코더는 항상 시작 토큰 <SOS> 으로 시작합니다.

text

코드 복사

<SOS> → token ID → embedding

그리고 이 임베딩이 디코더 **LSTM**의 첫 입력이 됩니다.

또한:

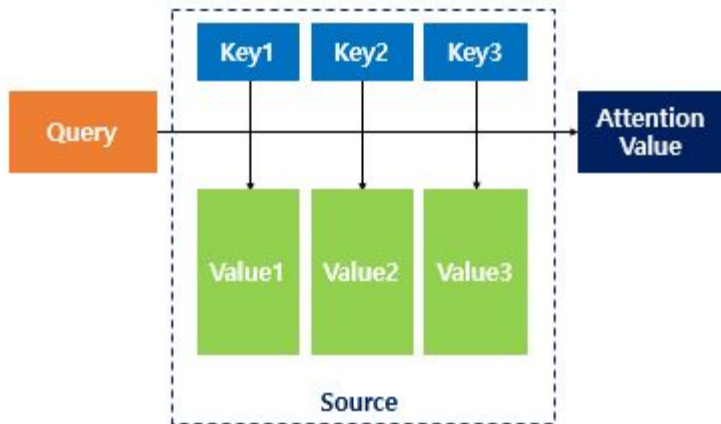
text

코드 복사

Decoder 초기 hidden state = Encoder의 Context Vector

Attention

- Seq2seq 인코더 단점
 - 첫째, 하나의 고정된 크기의 벡터에 모든 정보를 압축하려고 하니까 정보 손실이 발생합니다.
 - 둘째, RNN의 고질적인 문제인 기울기 소실(vanishing gradient) 문제가 존재합니다.
- 어텐션(Attention)의 아이디어



Attention

✓ 1. Attention이 등장한 이유 (문제부터 정확히)

기존 RNN 기반 Seq2Seq 구조:

text

📄 코드 복사

입력 문장 전체 → 마지막 h_T → Context Vector 1개 → 번역

✗ 한계

- 긴 문장일수록 초반 정보가 h_T 에 거의 남지 않음
- 모든 번역 단어가 같은 **Context Vector** 하나만 참고
- 즉,

“번역할 때마다 필요한 입력 부분만 골라 보지 못함”

Attention

✅ 2. Attention의 핵심 아이디어 (사람 번역 방식)

사람이 번역할 때:

```
text
```

📄 코드 복사

```
"I am a student"
```

```
→ "je"
```

```
→ "suis"
```

```
→ "étudiant"
```

각 단어를 만들 때마다:

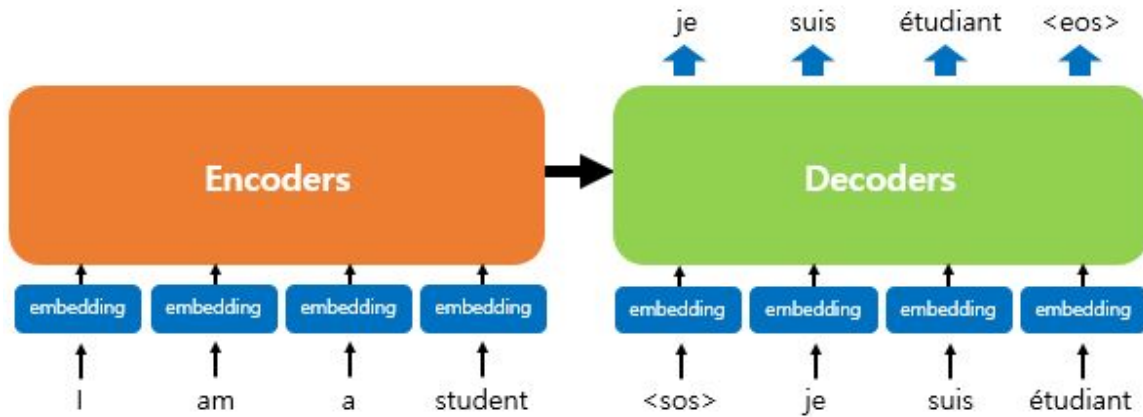
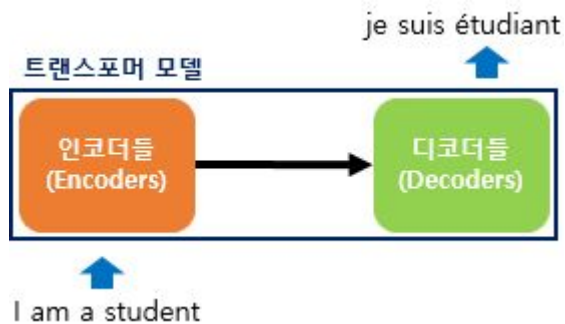
- 전체 문장을 다시 보지 않음
- 지금 필요한 단어에 해당하는 입력 부분만 집중해서 봄

✅ **Attention** = “출력 단어 하나를 만들 때, 입력 단어들 중 중요한 것들에 가중치를 주어 다시 참조하는 메커니즘”

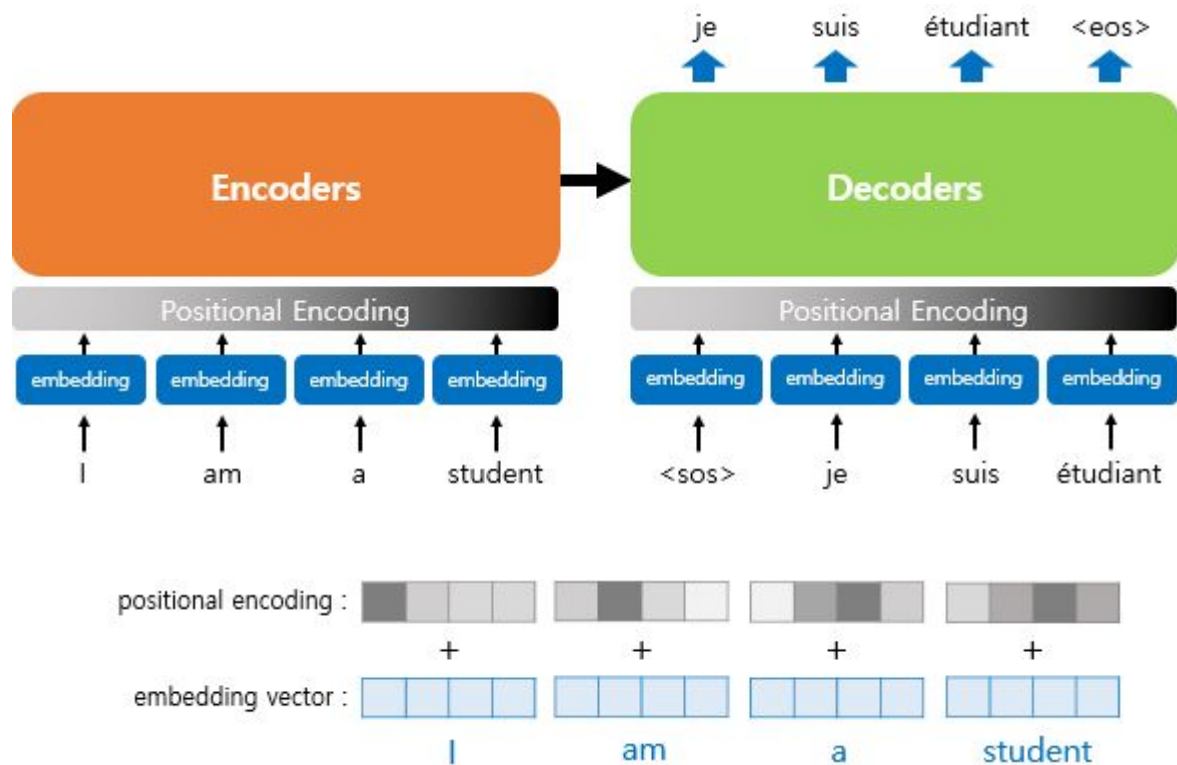
Seq2seq -> 트랜스포머

트랜스포머에 대해서 배우기 전에 기존의 **seq2seq**를 상기해봅시다. 기존의 **seq2seq** 모델은 인코더-디코더 구조로 구성되어 있었습니다. 여기서 인코더는 입력 시퀀스를 하나의 벡터 표현으로 압축하고, 디코더는 이 벡터 표현을 통해서 출력 시퀀스를 만들어냈습니다. 하지만 이러한 구조는 인코더가 입력 시퀀스를 하나의 벡터로 압축하는 과정에서 입력 시퀀스의 정보가 일부 손실된다는 단점이 있었고, 이를 보정하기 위해 어텐션이 사용되었습니다. 그런데 어텐션을 **RNN**의 보정을 위한 용도로서 사용하는 것이 아니라 어텐션만으로 인코더와 디코더를 만들어보면 어떨까요?

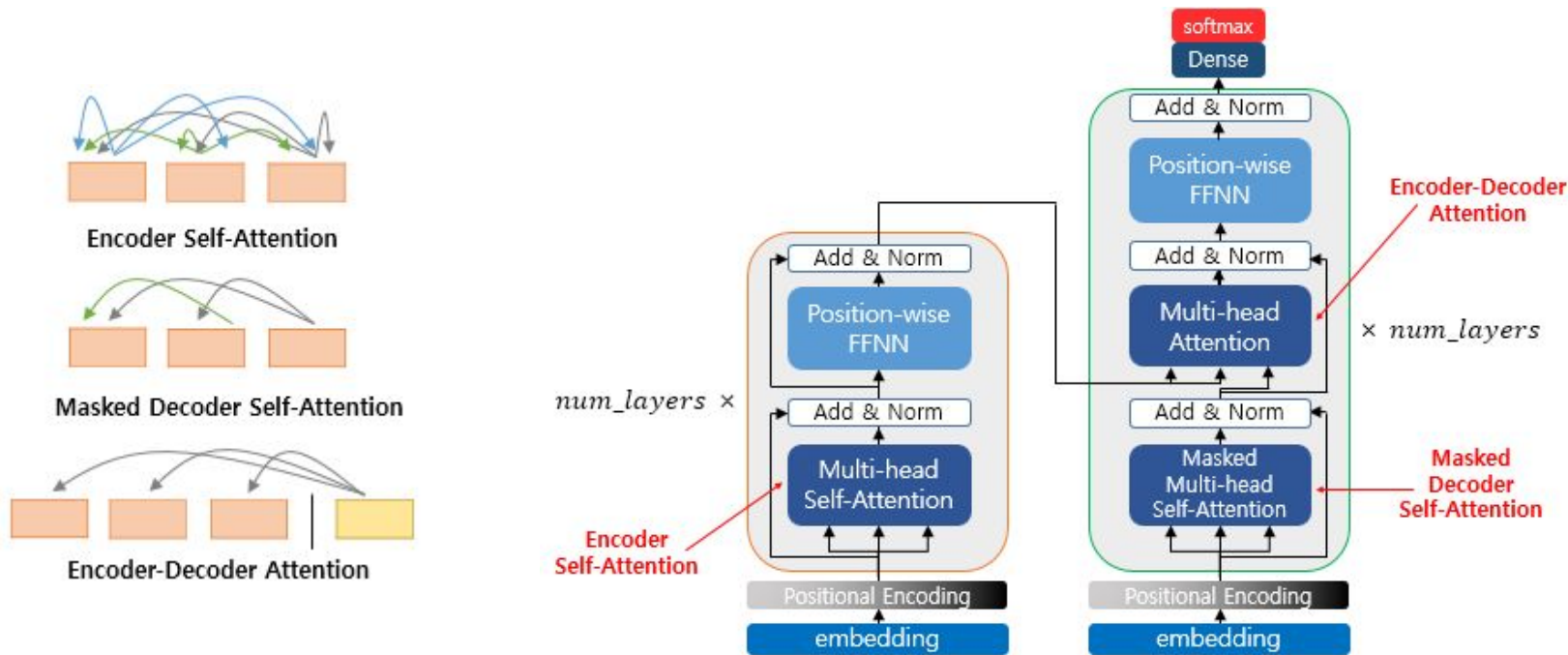
트랜스포머 구조



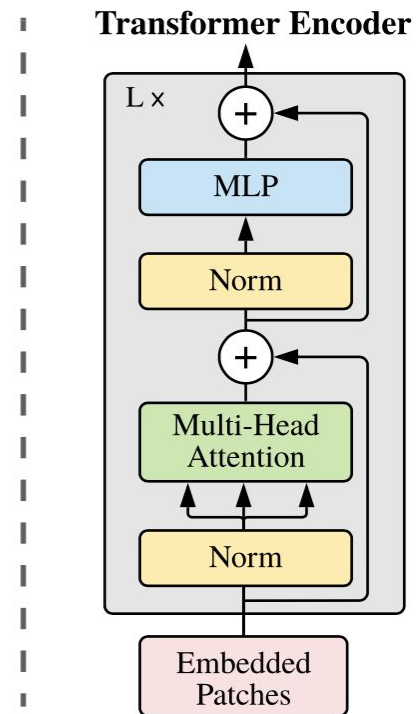
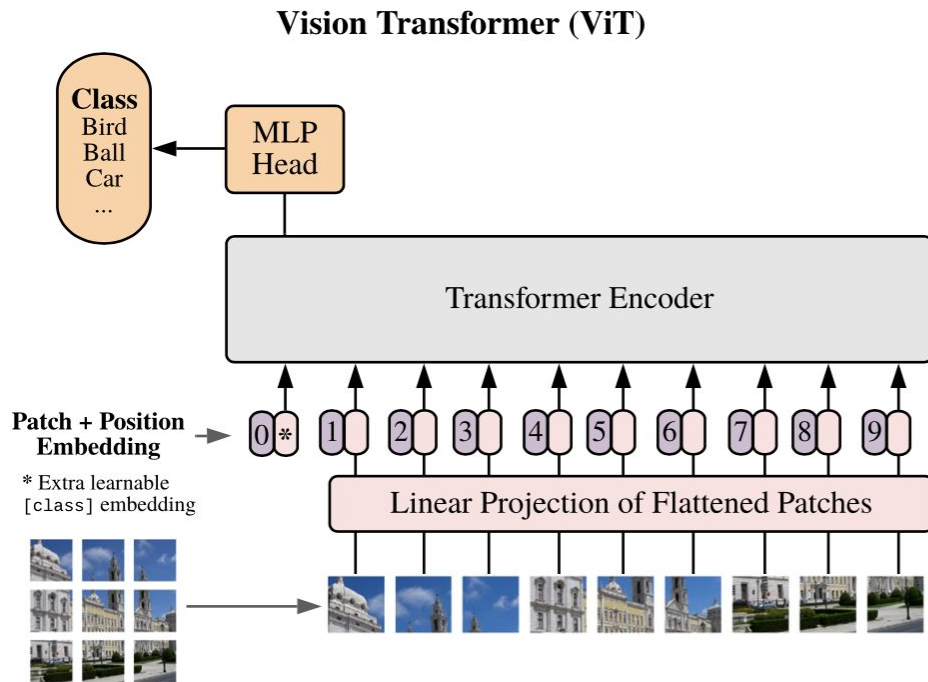
트랜스포머 Positional encoding



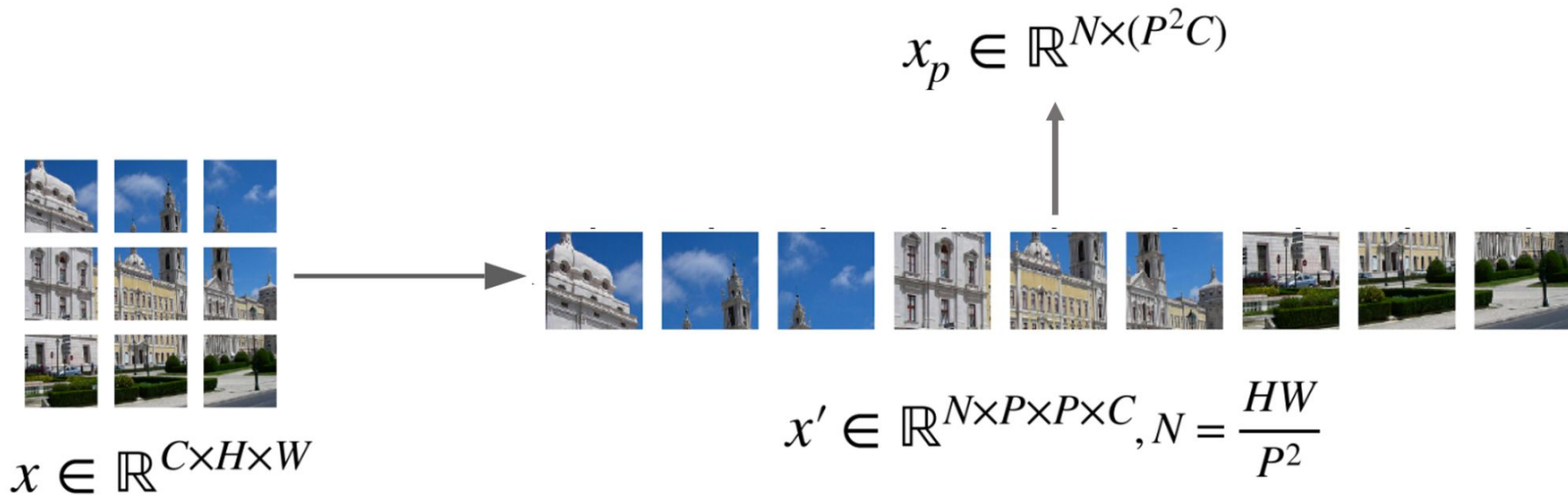
Self-attention



비전 트랜스포머

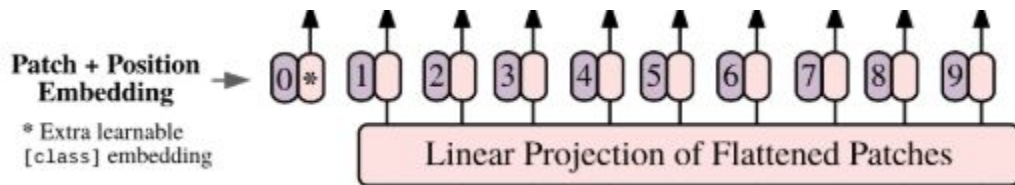


이미지 패치 만들기



이미지 패치 및 임베딩 단계

ViT에서는 이미지가 이렇게 처리됩니다:



CSS

코드 복사

이미지 (HxWxC)

→ $P \times P$ 크기로 패치 분할

→ 각 패치를 1차원 벡터로 펼침

→ Linear Projection

→ Patch Embedding

📌 그림 매칭

ViT 단계

그림 위치

패치 분할 + 선형변환

Input Embedding

위치 정보 추가

Positional Encoding ⊕

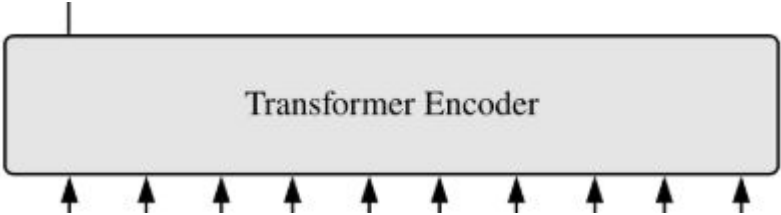
최종 입력

인코더 맨 아래

즉,

✅ “이미지 패치 → 임베딩 → 위치 인코딩” = 그림의 Input Embedding + Positional Encoding

Self-attention 인코딩



mathematica 코드 복사

Patch Embedding + Positional Encoding
→ Multi-Head Self-Attention
→ Add & Norm
→ MLP (Feed Forward)
→ Add & Norm
→ 이 블록을 N번 반복

📌 그림 매칭

ViT 연산

그림 위치

Self-Attention

Multi-Head Attention

잔차 연결 + 정규화

Add & Norm

반복

N ×

Linear Projection

✅ Linear Projection의 출력이 곧바로 “이미지 패치 임베딩(Patch Embedding)”입니다.

Vision Transformer에서는 다음이 완전히 동일한 단계입니다:

패치 → Linear Projection → Patch Embedding(Token)

아래에서 헛갈리지 않도록 수식, 텐서 형태, 실제 구현까지 정확히 정리해 드릴게요.

✅ 1 수식으로 보면 “출력 = 패치 임베딩”이 맞습니다

하나의 패치를 펼친 벡터를 x 라고 하면:

$$z = xW + b$$

- x : 한 패치의 픽셀 벡터 → $(P \times P \times C,)$
- W : 학습 가중치 → $(P \times P \times C, D)$
- z : Linear Projection 출력 → $(D,)$

이때:

✅ 이 z 벡터 **그 자체가 바로 “패치 임베딩”**입니다.

중간에 다른 변환은 없습니다.

ViT에서 트레이닝 되는 것들

구성 요소	학습됨?
패치 임베딩 가중치	✓
Q, K, V 가중치	✓
Attention Output Projection	✓
Attention 결과 벡터	✓ (학습 결과)
MLP 가중치	✓
[CLS] 토큰	✓
위치 임베딩	✓ 또는 ✗ (모델 설계에 따라 다름)

MLP

✓ ViT에서 MLP의 역할 한 줄 요약

Vision Transformer에서

✓ MLP = 인코더가 만든 "컨텍스트 벡터"를 받아서 최종 클래스로 변환하는 분류기(Decision Head)

1 전체 흐름을 한 줄로 정리

rust 📄 코드 복사

이미지 → 패치 임베딩 → Self-Attention 인코더 → 컨텍스트 벡터 → MLP → 클래스 확률

- 인코더의 출력 = 컨텍스트 벡터
- MLP의 출력 = 최종 예측 결과

MLP

2 MLP에 실제로 들어가는 입력은 무엇인가?

ViT에서는 인코더 출력 중에서 [CLS] 토큰 하나만 MLP로 들어갑니다.

mathematica

코드 복사

인코더 출력: (197, D)



CLS 토큰: (1, D)



MLP 입력

이 CLS 벡터는:

✓ 전체 이미지 정보를 압축한 전역 컨텍스트 벡터

입니다.

3 MLP의 정확한 역할 3가지

✓ ① 컨텍스트 → 의사결정 변환

- 인코더: "이 이미지가 어떤 구조를 가졌는가?"
- MLP: "그래서 이건 고양이인가? 자동차인가?"

CNN의 conv2D 출력과 트랜스포머의 self-attention의 출력인 context vector

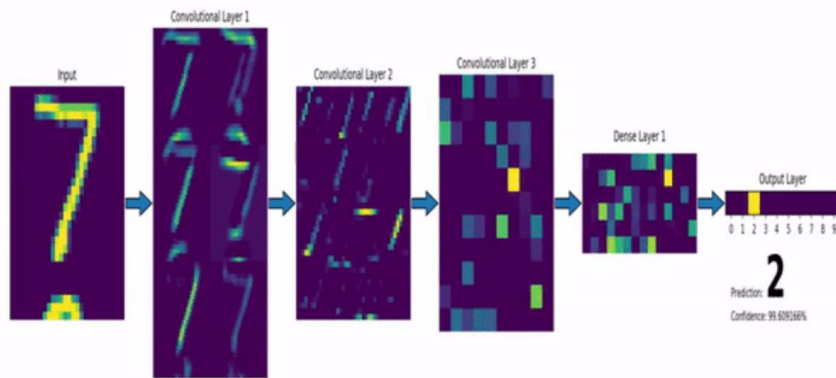
1 두 출력의 정체부터 한 줄로 정의

- Convolutional Neural Network Conv2D 출력

✓ 국소 영역(Local) 기반 특징지도(Feature Map)

- Self-Attention Context Vector

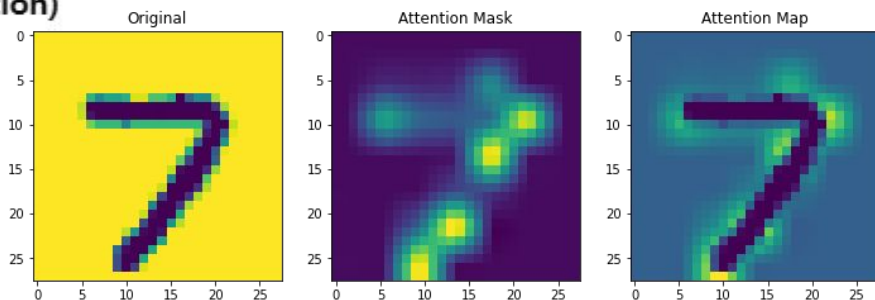
✓ 전역 관계(Global) 기반 의미 벡터(Token Representation)



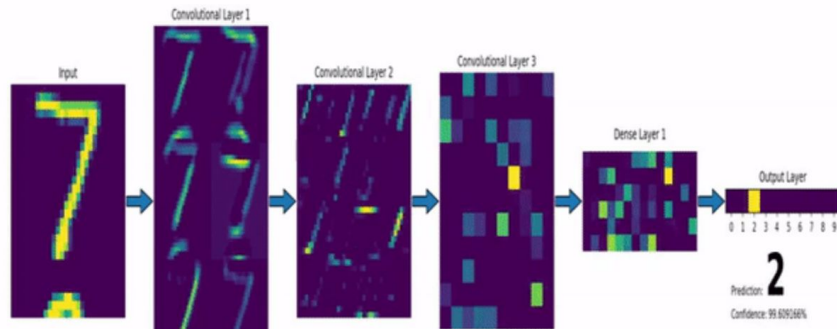
즉,

CNN은 “근처만 보며 특징을 쌓고”

트랜스포머는 “전체를 한 번에 보며 관계를 학습”합니다.



CNN feature map



층이 깊어지며 “의미가 어떻게 변하는가” 표로 정리

레이어	표현하는 정보	의미 수준
Input	픽셀 밝기	의미 없음
Conv1	엣지, 방향	저수준
Conv2	선 조합, 꺾임	중간 수준
Conv3	숫자 부분 구조	고수준
Dense	클래스 특징 벡터	개념
Output	숫자 확률	판단

ViT Attention map

✅ 이 MNIST Attention Map이 보여주는 핵심 메시지

1 CNN Feature Map과의 차이

- CNN:
 - 전체 윤곽선에 넓게 반응
- Self-Attention:
 - “이 숫자를 이 숫자로 구분하게 만드는 핵심 구조 포인트”만 선별 집중

2 Context Vector의 의미가 시각적으로 드러남

이 히트맵은 결국:

✅ Context Vector가 내부적으로 어떤 위치 정보를 가장 강하게 담고 있는지의 투영 결과

입니다.

즉,

수학적으로는 $Z \in \mathbb{R}^D$ 인 단순 벡터이지만,

그 안에는 “어느 위치가 중요한가”에 대한 전역 요약 정보가 들어 있고,

그걸 다시 픽셀 좌표로 역투영해서 색으로 표현한 것이 이 맵입니다.

